
CS771: Machine Learning, Minor Assignment 2

(2025–26 I)

Khushal Nikam	Prafull Joshi	Sarthak Dumbre	Vishal Junjare
251110043	251110056	251110064	251110085

1 Training GMM with K-components

1.1 Gaussian Mixture Model Formulation

In normal Gaussian classifiers, each class is modeled by a single Gaussian with some mean μ^c and covariance Σ^c . But this may not work properly as real data can have multiple subclusters per class. So, we will use K Gaussians instead of a single Gaussian.

$$p(\mathbf{x} \mid y = c) = \sum_{k=1}^K \pi_k^c \mathcal{N}(\mathbf{x} \mid \mu_k^c, \Sigma_k^c), \quad \sum_{k=1}^K \pi_k^c = 1. \quad (1.1)$$

Here:

- π_k^c – probability of choosing Gaussian k in class c
- μ_k^c – mean vector
- Σ_k^c – covariance matrix to understand shape and spread.
- K – total number of mixture components per class

Each class c is trained separately using the data points that belong to it.

1.2 Expectation–Maximization (EM) Derivation

1.2.1 Introduction

The objective is to estimate the parameters $\Theta^c = \{\pi_k^c, \mu_k^c, \Sigma_k^c\}_{k=1}^K$ for each class c , given the data points $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{N_c}\}$.

The log-likelihood of the observed data is given by:

$$\ell(\Theta^c) = \sum_{n=1}^{N_c} \log \left(\sum_{k=1}^K \pi_k^c \mathcal{N}(\mathbf{x}_n \mid \mu_k^c, \Sigma_k^c) \right).$$

Maximizing the log of a summation term is analytically intractable. To address this challenge, the Expectation–Maximization (EM) algorithm is employed. The EM algorithm provides a general framework for obtaining maximum likelihood estimates in the presence of missing or latent (un-observed) variables. In particular, it assumes that certain variables are latent and iteratively refines parameter estimates using their expected values. A detailed derivation of the EM procedure can be found in (1) (2).

1.2.2 Complete-Data Log-Likelihood

If the latent component assignments $\{z_n\}$ were known, the complete-data log-likelihood would take the form:

$$\mathcal{L}_{\text{complete}}(\Theta^c) = \sum_{n=1}^{N_c} \sum_{k=1}^K \mathbb{I}[z_n = k] \left(\log \pi_k^c + \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k^c, \boldsymbol{\Sigma}_k^c) \right).$$

Since z_n is not observed, the EM algorithm replaces it with its expectation under the current parameter estimates, effectively computing the expected complete-data log-likelihood.

1.2.3 E-step

In the Expectation step (E-step), we compute the posterior probability that data point $\mathbf{x}_n$ belongs to component k :

$$\gamma_{nk}^c = P(z_n = k \mid \mathbf{x}_n, \Theta^{c,\text{old}}) \quad (1.2)$$

$$= \frac{\pi_k^{c,\text{old}} \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k^{c,\text{old}}, \boldsymbol{\Sigma}_k^{c,\text{old}})}{\sum_{j=1}^K \pi_j^{c,\text{old}} \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_j^{c,\text{old}}, \boldsymbol{\Sigma}_j^{c,\text{old}})}. \quad (1.3)$$

The term γ_{nk}^c represents the *soft assignment* of $\mathbf{x}_n$ to component k , indicating the degree of responsibility that component k has for generating that observation.

1.2.4 M-step: Parameter Updates

In the Maximization step (M-step), the parameters are re-estimated by maximizing the expected complete-data log-likelihood:

$$Q(\Theta^c, \Theta^{c,\text{old}}) = \sum_{n,k} \gamma_{nk}^c \left(\log \pi_k^c + \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k^c, \boldsymbol{\Sigma}_k^c) \right).$$

Effective Counts: The effective number of samples assigned to each component k is given by:

$$N_k^c = \sum_{n=1}^{N_c} \gamma_{nk}^c$$

Mixture Weights: Maximizing $\sum_k N_k^c \log \pi_k^c$ subject to $\sum_k \pi_k^c = 1$ yields:

$$\pi_k^{c,\text{new}} = \frac{N_k^c}{N_c}$$

Means: Differentiating Q with respect to $\boldsymbol{\mu}_k^c$ and equating to zero gives:

$$\boldsymbol{\mu}_k^{c,\text{new}} = \frac{1}{N_k^c} \sum_{n=1}^{N_c} \gamma_{nk}^c \mathbf{x}_n$$

Covariance Matrices: Optimizing for $\boldsymbol{\Sigma}_k^c$ similarly gives:

$$\boldsymbol{\Sigma}_k^{c,\text{new}} = \frac{1}{N_k^c} \sum_{n=1}^{N_c} \gamma_{nk}^c (\mathbf{x}_n - \boldsymbol{\mu}_k^{c,\text{new}})(\mathbf{x}_n - \boldsymbol{\mu}_k^{c,\text{new}})^\top + \epsilon \mathbf{I}$$

The small regularization term $\epsilon \mathbf{I}$ is added for numerical stability, as commonly done in practical implementations (3).

1.2.5 Log-Likelihood and Convergence

After each iteration of the EM algorithm, the data log-likelihood is recomputed as:

$$\ell(\Theta^c) = \sum_{n=1}^{N_c} \log \left(\sum_{k=1}^K \pi_k^c \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k^c, \boldsymbol{\Sigma}_k^c) \right).$$

The algorithm is deemed to have converged when the change in $\ell(\Theta^c)$ between successive iterations falls below a predefined threshold, indicating stability of the parameter estimates.

1.3 Classification using Trained Class-Conditional GMMs

Once each class c has its GMM parameters learned, we can classify a new $\mathbf{x}$ using Bayes' rule:

$$p(y = c \mid \mathbf{x}) = \frac{p(y = c) p(\mathbf{x} \mid y = c)}{\sum_{c'} p(y = c') p(\mathbf{x} \mid y = c')}, \quad (1.4)$$

$$p(\mathbf{x} \mid y = c) = \sum_{k=1}^K \pi_k^c \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k^c, \boldsymbol{\Sigma}_k^c). \quad (1.5)$$

We then predict

$$\hat{y} = \arg \max_c [\log p(y = c) + \log p(\mathbf{x} \mid y = c)].$$

1.4 Algorithm Summary (for one class)

1. Initialize $\{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}$.
2. Repeat until convergence:
 - (a) E-step: Compute γ_{nk} using current parameters.
 - (b) M-step: Update $\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k$.
 - (c) Check change in log-likelihood.
3. After training for all classes, classify new data via Bayes' rule.

2 Inference with a GMM-based Generative Multi-class Classifier

Inference Objective: Once we have trained a Gaussian Mixture Model (GMM) for each class, the next step is to perform inference or testing on a new unseen test point $\mathbf{x}_{\text{test}}$. The goal is to compute the posterior class probabilities $p(y = c \mid \mathbf{x}_{\text{test}})$ for all classes and then predict the most likely class.

1. Parameters known after training:

From the EM training (done separately for each class c), we already have the learned parameters:

$$\Theta^c = \{\pi_{c,k}, \boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k}\}_{k=1}^K, \quad \text{and class prior } \rho_c = p(y = c).$$

Here $\pi_{c,k}$ are the mixture weights, $\boldsymbol{\mu}_{c,k}$ are means, and $\boldsymbol{\Sigma}_{c,k}$ are covariance matrices for the k -th Gaussian component of class c .

2. Compute class-conditional likelihoods:

For a new test point $\mathbf{x}$, the probability of observing $\mathbf{x}$ under class c is given by:

$$p(\mathbf{x} \mid y = c) = \sum_{k=1}^K \pi_{c,k} \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k}). \quad (2.1)$$

Expanding the Gaussian term:

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_{c,k}, \boldsymbol{\Sigma}_{c,k}) = \frac{1}{(2\pi)^{D/2} |\boldsymbol{\Sigma}_{c,k}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_{c,k})^\top \boldsymbol{\Sigma}_{c,k}^{-1}(\mathbf{x} - \boldsymbol{\mu}_{c,k})\right). \quad (2.2)$$

This gives the likelihood of the test point for each class.

3. Apply Bayes' rule to compute class posteriors:

Using the class prior $\rho_c = p(y = c)$, the posterior probability for each class is:

$$p(y = c \mid \mathbf{x}) = \frac{\rho_c p(\mathbf{x} \mid y = c)}{\sum_{c'=1}^C \rho_{c'} p(\mathbf{x} \mid y = c')}. \quad (2.3)$$

Since all quantities on the right are known or can be computed, we can directly evaluate $p(y = c \mid \mathbf{x})$ for every class.

4. Prediction rule (MAP decision):

The predicted class label for the test point is the one with the maximum posterior probability:

$$\hat{y} = \arg \max_c [\rho_c p(\mathbf{x} | y = c)]. \quad (2.4)$$

In practice, we work in log-space to avoid numerical underflow:

$$\hat{y} = \arg \max_c [\log \rho_c + \log p(\mathbf{x} | y = c)]. \quad (2.5)$$

5. Log-space evaluation for numerical stability:

Because each $p(\mathbf{x} | y = c)$ involves exponentials, direct computation can lead to underflow. So we compute the *log-likelihood* instead:

$$\ell_{c,k}(\mathbf{x}) = \log \pi_{c,k} - \frac{D}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma_{c,k}| - \frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_{c,k})^\top \Sigma_{c,k}^{-1} (\mathbf{x} - \boldsymbol{\mu}_{c,k}), \quad (2.6)$$

$$\log p(\mathbf{x} | y = c) = \log \sum_{k=1}^K \exp(\ell_{c,k}(\mathbf{x})). \quad (2.7)$$

To compute this safely, use the log-sum-exp (LSE) trick:

$$\log p(\mathbf{x} | y = c) = m_c + \log \sum_{k=1}^K \exp(\ell_{c,k}(\mathbf{x}) - m_c), \quad m_c = \max_k \ell_{c,k}(\mathbf{x}).$$

This ensures numerical stability.

6. Component-wise posterior probabilities (optional):

We can also find the posterior probability that the point $\mathbf{x}$ belongs to the k -th component within class c :

$$p(z = k | \mathbf{x}, y = c) = \frac{\pi_{c,k} \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_{c,k}, \Sigma_{c,k})}{\sum_{j=1}^K \pi_{c,j} \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_{c,j}, \Sigma_{c,j})}. \quad (2.8)$$

These “responsibilities” at test time indicate which sub-Gaussian inside a class most likely produced the sample.

7. Final prediction summary:

- (a) For each class c :
 - i. Compute $p(\mathbf{x} | y = c)$ using Eq. (2.1) (preferably in log-space).
 - ii. Multiply by class prior ρ_c to get joint likelihood $\rho_c p(\mathbf{x} | y = c)$.
- (b) Normalize across classes using Eq. (2.3) to get posteriors $p(y = c | \mathbf{x})$.
- (c) Choose $\hat{y}$ using Eq. (2.4).

In summary, during inference, the classifier evaluates how likely each class-specific GMM is to have generated the given test sample. The class model that assigns the highest likelihood (or equivalently, the highest posterior probability) to the observation is selected as the predicted class.

3 Major Code Modifications for GMM-based Generative Classification

The original lecture implementation modeled each class c in the MNIST dataset as a single multivariate Gaussian distribution $\mathcal{N}(\mu_c, \Sigma_c)$. To better capture intra-class variability (e.g., different handwriting styles of the same digit), the model was extended to use a **Gaussian Mixture Model (GMM)** representation for each class.

Each class is now represented as a mixture of K Gaussian components:

$$P(\mathbf{x} | y = c) = \sum_{k=1}^K \pi_{ck} \mathcal{N}(\mathbf{x} | \mu_{ck}, \Sigma_{ck})$$

where π_{ck} are the mixture weights that satisfy $\sum_{k=1}^K \pi_{ck} = 1$.

3.1 Extension from Single Gaussian to Mixture Model

In the lecture code, each class was represented by one mean vector μ_c and one covariance matrix Σ_c . In this implementation, these parameters are replaced by sets of means, covariances, and mixture weights: $\{\mu_{ck}\}_{k=1}^K$, $\{\Sigma_{ck}\}_{k=1}^K$, and $\{\pi_{ck}\}_{k=1}^K$. This modification allows multiple clusters to represent distinct data subregions within the same class.

3.2 Revised E-step and M-step Implementations

The EM algorithm was implemented to estimate the GMM parameters for each class. The following functions were modified:

(1) `doEStep()` :

This function computes the responsibilities γ_{nk}^c , which represent the probability that sample $\mathbf{x}_n$ was generated by component k . Mathematically:

$$\gamma_{nk}^c = \frac{\pi_k^c \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k^c, \boldsymbol{\Sigma}_k^c)}{\sum_{j=1}^K \pi_j^c \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_j^c, \boldsymbol{\Sigma}_j^c)}.$$

Changes in the `doEStep()` function:

- **Mixture weights incorporation:** Instead of treating all clusters equally, the responsibilities γ_{nk} are now weighted by the prior probabilities π_k , using the relation:

$$\gamma_{nk} \propto \pi_k \exp\left(-\frac{1}{2}D_{nk}^2\right),$$

where D_{nk}^2 denotes the squared Euclidean distance between data point $\mathbf{x}_n$ and mean $\boldsymbol{\mu}_k$.

- **Normalization:** Responsibilities are normalized across all K components for each data point:

$$\gamma_{nk} = \frac{\pi_k \exp\left(-\frac{1}{2}D_{nk}^2\right)}{\sum_{j=1}^K \pi_j \exp\left(-\frac{1}{2}D_{nj}^2\right)}.$$

- **Default weights:** If π_{vals} is not provided, equal mixture weights are assumed:

$$\pi_k = \frac{1}{K}, \quad \forall k \in \{1, \dots, K\}.$$

(2) `doMStep()` :

- **Generalization to full GMM M-step:** The new implementation performs a complete maximization step of the Gaussian Mixture Model (GMM), updating not only the means $\boldsymbol{\mu}_k$, but also the covariances $\boldsymbol{\Sigma}_k$ and mixture weights π_k . The previous version only updated the means.
- **Updated mathematical formulation:** The new code implements the standard GMM M-step equations for each class (1) (2):

$$N_k = \sum_{n=1}^N \gamma_{nk}, \quad \pi_k = \frac{N_k}{N}, \quad \boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} \mathbf{x}_n,$$

$$\boldsymbol{\Sigma}_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (\mathbf{x}_n - \boldsymbol{\mu}_k)(\mathbf{x}_n - \boldsymbol{\mu}_k)^\top + \epsilon \mathbf{I}.$$

- **Covariance estimation added:** The old version did not compute or update the covariance matrices $\boldsymbol{\Sigma}_k$. The new version estimates them using the weighted scatter of data points around their respective means.
- **Mixture weights π_k :** The mixture weights are now explicitly computed as $\pi_k = N_k/N$, allowing each component to have a proportional contribution to the mixture model.
- **Numerical stability:** A small regularization term $\epsilon \mathbf{I}$ is added to each covariance matrix to prevent singularities. (3)
- **Cluster fallback:** For clusters with insufficient data (i.e., $N_k \leq 1$), the old parameter values $\boldsymbol{\mu}_k^{\text{old}}$ and $\boldsymbol{\Sigma}_k^{\text{old}}$ are retained, and their mixture weights are set to $1/K$.

(3) `doEMGMM()` :

- **Inclusion of mixture weights π_k :** The new version explicitly introduces mixture weights as model parameters, initialized as $\pi_k = 1/K$. These weights are now updated during each M-step, allowing for a full Gaussian Mixture Model (GMM) implementation.
- **Modified E-step call:** The function now calls the updated `doEStep` that accepts π_k as an additional argument. This ensures that responsibilities γ_{nk} are computed using weighted Gaussian components.
- **Enhanced M-step:** The updated `doMStep` is invoked to jointly update means μ_k , covariances Σ_k , and mixture weights π_k according to the standard GMM update rules:

$$N_k = \sum_{n=1}^N \gamma_{nk}, \quad \pi_k = \frac{N_k}{N}, \quad \mu_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} \mathbf{x}_n,$$

$$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (\mathbf{x}_n - \mu_k)(\mathbf{x}_n - \mu_k)^\top + \epsilon \mathbf{I}.$$

- **Removal of redundant covariance update:** The final loop that recomputed Σ_k after all EM iterations in the old code has been removed. Covariances are now correctly and consistently updated during each M-step.
- **Updated return values:** The function now returns four parameters instead of three:

$$(\mu_{\text{Vals}}, \Sigma_{\text{Vals}}, \mathbf{q}_{\text{Vals}}, \pi_{\text{Vals}}),$$

representing the updated means, covariances, responsibilities, and mixture weights.

- **Cleaner and more efficient structure:** The overall implementation is more compact and avoids redundant computation, ensuring the EM loop fully reflects the theoretical formulation of the Gaussian Mixture Model.

3.3 Implementation Modifications for Multi-Component Class-Conditional GMMs

In the lecture-provided implementation, each class-conditional distribution was modeled using a **single Gaussian**, i.e., $K = 1$. The parameters — mean vector μ_c and covariance matrix Σ_c — were estimated in a **supervised Maximum Likelihood Estimation (MLE)** manner, directly using all training samples belonging to class c . Specifically, for each class, the code computed the empirical mean and covariance of its feature vectors, resulting in a unimodal Gaussian representation per class.

To extend this model into a **Gaussian Mixture Model (GMM)** framework, we modified the `learnClassConditionalDist()` function into a new function named `learnClassConditionalDist_GMM()`. The key changes introduced are as follows:

1. **Multiple Components per Class ($K > 1$):** Instead of fitting a single Gaussian per class, we now fit a mixture of K Gaussian components for each class c . This allows each class to capture **multi-modal distributions** within its data (e.g., variations in how the same digit is written).
2. **Unsupervised Parameter Estimation via EM Algorithm:** In the original implementation, parameters (μ_c, Σ_c) were estimated using analytical MLE formulas. In our modified version, parameter estimation is performed using the **Expectation-Maximization (EM)** algorithm implemented in the `doEMGMM()` function. This algorithm iteratively refines the component means μ_{ck} , covariances Σ_{ck} , and mixture weights π_{ck} until convergence.
3. **Initialization using k-means++:** To initialize the EM algorithm effectively, we have used lecture code of a **k-means++ initialization** scheme via the `doKMPPInitGreedy()`.
4. **Class Prior Retention:** The **class prior probabilities** $P(y = c)$ are still computed as the relative frequency of each class in the training set, ensuring consistency with the generative classification framework.
5. **Model Representation:** The returned model now stores parameters for all components of all classes in the form:

$$(\mu_{ck}, \Sigma_{ck}, c, k, \pi_{ck}, P(y = c))$$

This allows subsequent inference code to marginalize over all mixture components of each class during likelihood computation.

6. **Scalability to Different K Values:** The function was designed to allow easy variation of the number of components K , enabling systematic experiments with $K = 1, 2, 5, 10, 15, 20$, etc., to study the effect of mixture complexity on model accuracy.

3.4 Updated Prediction Function for GMMs

Changes in the `predictClassScores()` and `predictGen()` functions:

- **Function names and purpose:** The old functions `predictClassScores()` and `predictGen()` were designed for class-conditional densities modeled by a *single Gaussian per class*. The updated versions extend this to handle *Gaussian Mixture Models (GMMs)* with multiple components per class.
- **Input parameters:** The updated function takes lists of mixture parameters:

$$(\mu_{ck}, \Sigma_{ck}, \pi_{ck}), \quad k = 1, \dots, K$$

instead of a single mean μ_c and covariance Σ_c . It also takes the class prior $p = P(Y = c)$ as before. The optional mask argument (for feature selection) remains unchanged.

- **Computation over mixture components:** Instead of evaluating a single Gaussian log-likelihood, the updated function iterates over all mixture components k of the class:

$$\log P(\mathbf{x}_n | Y = c) = \log \left(\sum_{k=1}^K \pi_{ck} \mathcal{N}(\mathbf{x}_n; \mu_{ck}, \Sigma_{ck}) \right).$$

The old version computed only

$$\log P(\mathbf{x}_n | Y = c) = \log \mathcal{N}(\mathbf{x}_n; \mu_c, \Sigma_c).$$

- **Full log-likelihood computation:** The updated version explicitly includes the Gaussian normalization term

$$-\frac{1}{2} [D \log(2\pi) + \log |\Sigma_{ck}| + (\mathbf{x}_n - \mu_{ck})^T \Sigma_{ck}^{-1} (\mathbf{x}_n - \mu_{ck})],$$

whereas the old version omitted the constant term $-\frac{1}{2} D \log(2\pi)$.

- **Handling near-singular covariances:** As before, the pseudoinverse (`lin.pinv`) and `slogdet` are used for covariance inversion and determinant computation. If the determinant is non-positive, the log-determinant term is set to zero to maintain numerical robustness.
- **Posterior computation:** After obtaining the class-conditional log-likelihood $\log P(\mathbf{x}_n | Y = c)$, the posterior log-probability is computed as:

$$\log P(Y = c | \mathbf{x}_n) = \log P(\mathbf{x}_n | Y = c) + \log P(Y = c).$$

- **Updated general prediction function:** The new `predictGen_GMM()` loops through all class models, each containing multiple mixture components, and computes posterior scores for each class using the new GMM-based scoring function. The predicted label is then obtained as:

$$\hat{y} = \arg \max_c \log P(Y = c | \mathbf{x}_n).$$

3.5 Accuracy Evaluation and Visualization

The model was trained for multiple values of $K \in \{1, 2, 3, 5, 10, 15, 20\}$. For each K , both training and test accuracies were recorded using `predictGen_GMM()` on X_{train} and X_{test} , respectively. Finally, a curve of accuracy vs. K was plotted to visualize the impact of model complexity.

3.6 Summary of Key Functional Changes

- Added EM-based parameter estimation through new functions `doEStep()`, `doMStep()`, and `doEMGMM()`.
- Modified `learnClassConditionalDist()` to support K -component mixtures per class
- Rewrote prediction logic to compute log-sum of mixture likelihoods per class
- Added support for evaluating accuracy and visualizing performance variation with K .

Overall, these modifications extend the single-Gaussian generative classifier into a multi-modal, data-driven GMM classifier capable of modeling the diverse handwriting patterns present in the MNIST dataset.

3.7 Effect of Number of GMM Components (K) on Accuracy

To analyze the effect of model complexity, we varied the number of Gaussian components K used per class in the Gaussian Mixture Model (GMM). Each of the 10 digit classes was represented using the same value of K , i.e., if $K = 5$, then each class-specific GMM contained 5 Gaussian components.

The experiment was conducted for $K = 1, 2, 3, 5, 10, 15, 20$, and the resulting training and testing accuracies were recorded. The x-axis represents the number of Gaussian components (K), while the y-axis shows the corresponding training and testing accuracies. All testing accuracies were evaluated on the uncensored test images.

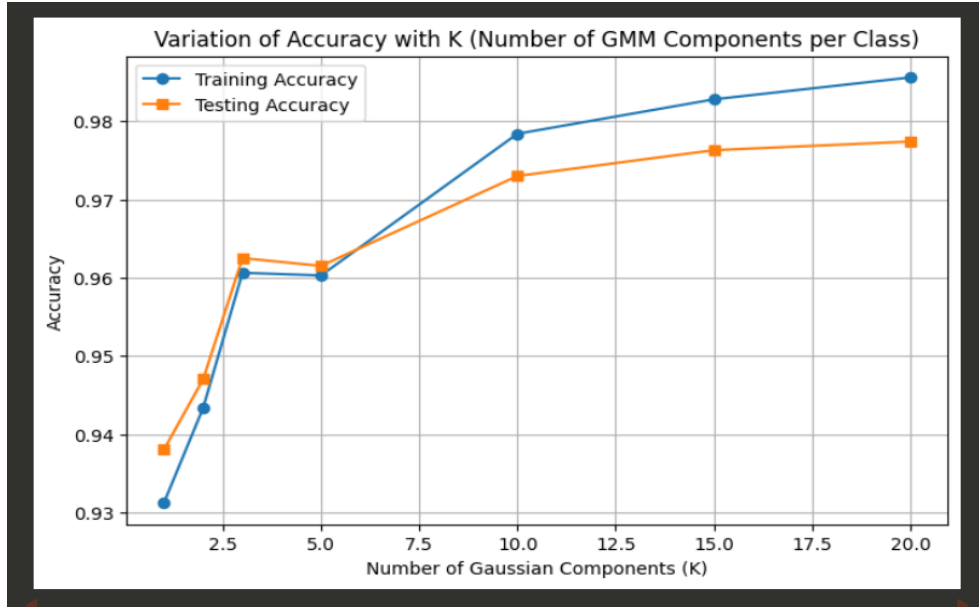


Figure 1: Variation of Training and Testing Accuracy with Number of GMM Components per class on uncensored test images.

3.7.1 Observation

- From Figure 1, it is evident that as the number of mixture components K increases, both training and testing accuracies improve initially. The training accuracy continues to rise steadily as K grows, reflecting the model's enhanced ability to capture intra-class variations. As K increases, the model becomes more expressive, leading to higher training accuracy and initially improving test accuracy, but beyond a certain point, training time grows significantly and test accuracy saturates or even declines due to overfitting.
- Table 1 presents the prediction accuracy (%) of the model for various values of the regularization parameter ϵ and the number of Gaussian components K . As ϵ increases, accuracy

gradually decreases, indicating that larger regularization values may oversmooth the covariance estimates. Conversely, increasing K generally improves accuracy, showing that a higher number of mixture components allows for better modeling of class distributions.

ϵ	$K = 1$	$K = 2$	$K = 3$	$K = 5$	$K = 10$
$1e^{-1}$	93.83	94.97	95.57	96.45	97.38
$1e^{-2}$	92.36	92.98	94.25	95.19	96.89
$1e^{-3}$	88.27	89.89	91.00	92.68	94.19
$1e^{-4}$	83.73	86.74	87.83	89.08	91.32
$1e^{-5}$	80.04	82.65	84.33	87.05	88.6
0	82.09	84.55	83.86	77.12	81.00

Table 1: Prediction accuracy (%) after tuning ϵ and K Values

Note: The Idea of using a regularization term while estimating covariance is taken from (3)

4 Generative Classification and Reconstruction with Missing Pixels

We extended the lecture code to perform **generative classification** and **image reconstruction** in the presence of missing pixels using a Gaussian Mixture Model (GMM) framework.

The procedure follows the same image censoring strategy as in the original implementation; that is, the method `sensorImages()` is used without modification to remove a fixed central region of the image. The censored regions simulate missing data, and the model must classify and reconstruct images based only on the visible pixels.

The original `reconstructImages()` function was designed for a single Gaussian per class, whereas the updated version extends it to handle a Gaussian Mixture Model (GMM) with multiple components per class. The major modifications are as follows:

1. Support for Multiple Gaussian Components:

- The old version extracted a single mean and covariance (μ, Σ) for each class.
- The new version unpacks a list of parameters from the model:

$$(\mu_1, \Sigma_1), (\mu_2, \Sigma_2), \dots, (\mu_K, \Sigma_K),$$

along with mixture weights $\pi_1, \pi_2, \dots, \pi_K$.

- This allows the reconstruction to incorporate contributions from all K mixture components.

2. Weighted Combination of Reconstructions:

- In the old version, reconstruction of missing pixels was done using the conditional mean of a single Gaussian:

$$\mathbf{x}_{\text{miss}} = \mu_{\text{miss}} + \Sigma_{\text{miss,obs}} \Sigma_{\text{obs,obs}}^{-1} (\mathbf{x}_{\text{obs}} - \mu_{\text{obs}}).$$

- The new version computes the above conditional expectation for each Gaussian component k :

$$\mathbf{x}_{\text{miss}}^{(k)} = \mu_{\text{miss}}^{(k)} + \Sigma_{\text{miss,obs}}^{(k)} (\Sigma_{\text{obs,obs}}^{(k)})^{-1} (\mathbf{x}_{\text{obs}} - \mu_{\text{obs}}^{(k)}).$$

- Then, reconstructions from all components are averaged using the mixture weights:

$$\mathbf{x}_{\text{miss}} = \frac{\sum_{k=1}^K \pi_k \mathbf{x}_{\text{miss}}^{(k)}}{\sum_{k=1}^K \pi_k}.$$

3. Loop over Mixture Components:

- A new loop over $k = 1, 2, \dots, K$ was introduced to compute and combine per-component reconstructions.

- Temporary variables `recon_total` and `weight_total` accumulate the weighted sum and total weights, respectively.

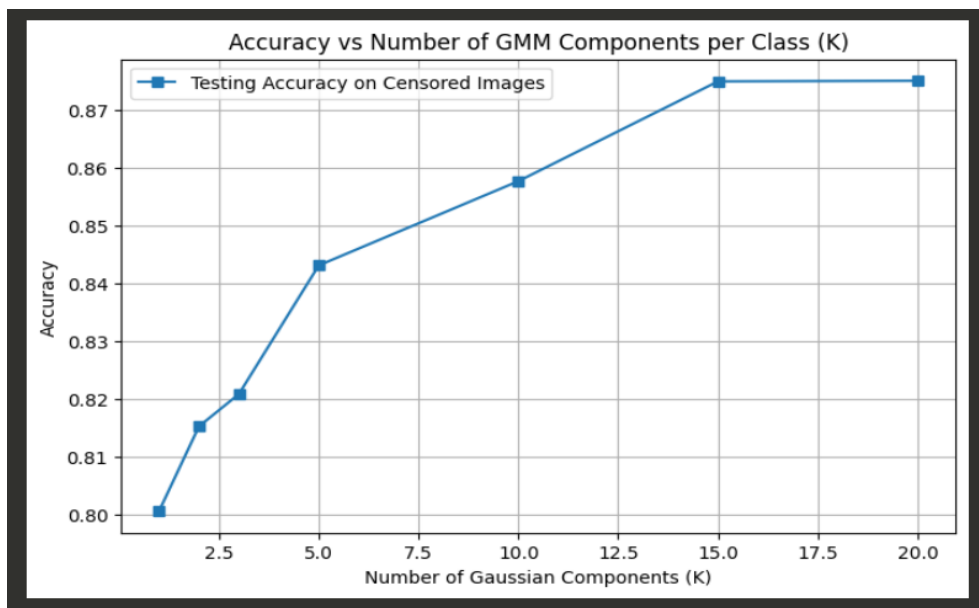


Figure 2: Variation of test accuracy on censored images with number of GMM components K .

4.1 Classification and Reconstruction under Missing Pixels:

4.1.1 Correctly Classified Images

Figure 3 shows sample test images where the model predicted the correct digit label even after censoring a portion of the image.

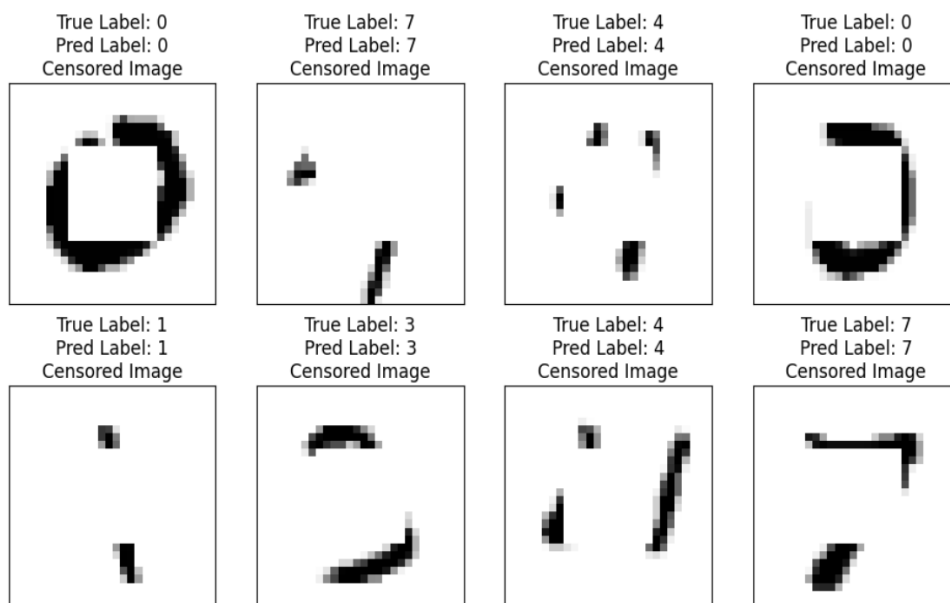


Figure 3: Examples of correctly classified test images with missing pixels.

4.1.2 Incorrectly Classified Images

Figure 4 shows examples where the classifier failed to predict the correct class.

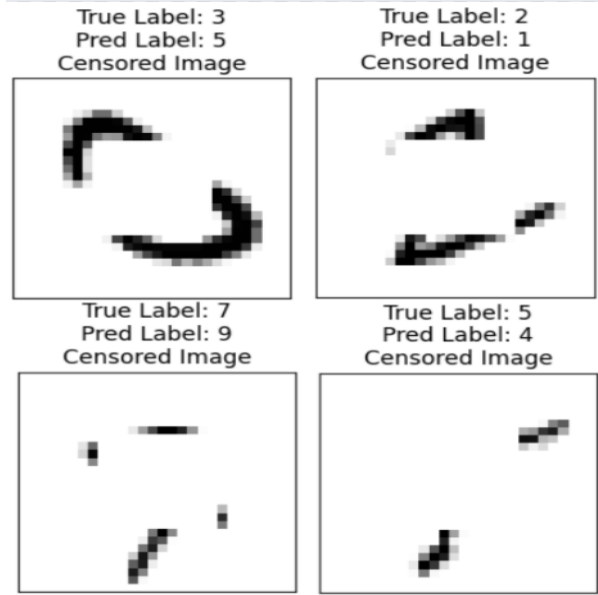


Figure 4: Examples of incorrectly classified test images with missing pixels.

4.1.3 Successful Reconstructions

Figure 5 shows examples where missing regions were correctly reconstructed by the GMM model.

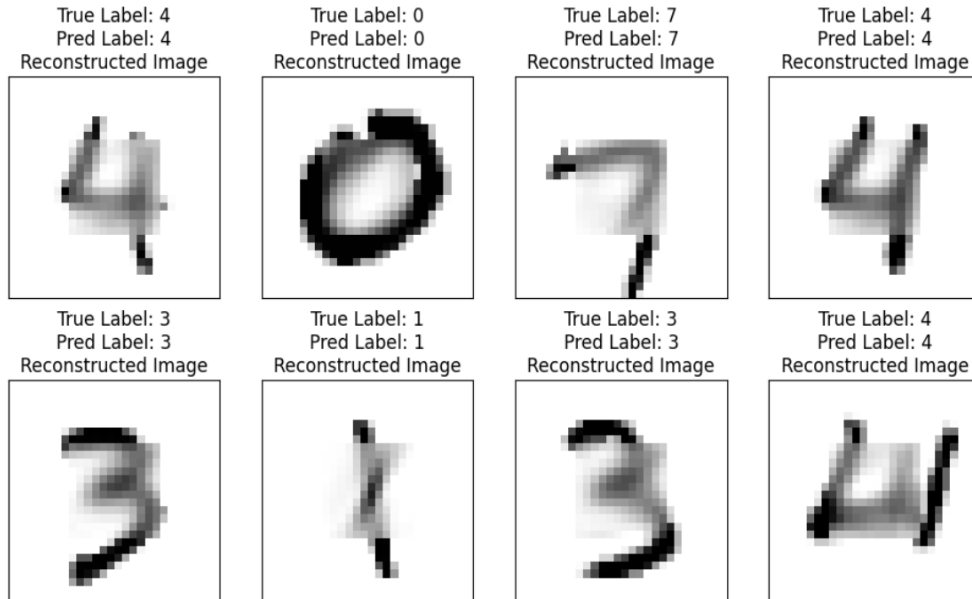


Figure 5: Examples of successful reconstructions of censored test images.

4.1.4 Unsuccessful Reconstructions

Figure 6 presents examples where the reconstructions failed to recover the correct digit structure, often due to incorrect class prediction or insufficient mixture modeling.

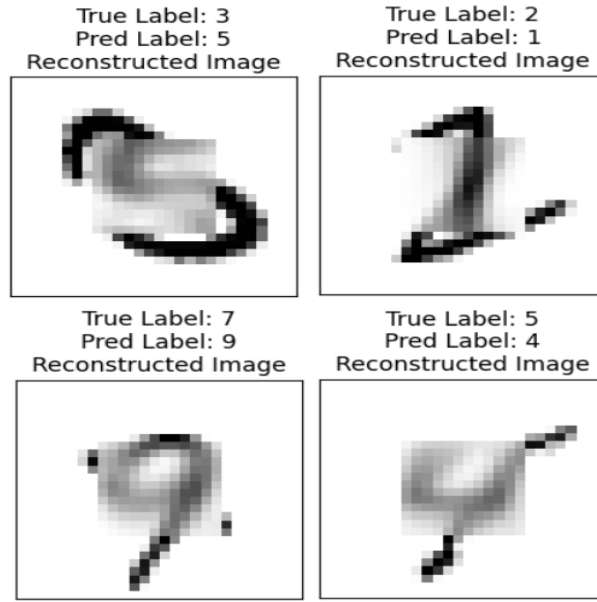


Figure 6: Examples of unsuccessful reconstructions on censored test images.

5 Bonus: Effect of Varying Number of Gaussian Components per Class

1. **Setup:** Two generative classifiers were trained for comparison:

- **Fixed-K model:** Each digit class modeled using $K = 10$ Gaussian components.
- **Variable-K model:** Each digit is assigned a custom number of components : $K = \{0:3, 1:2, 2:8, 3:10, 4:5, 5:9, 6:6, 7:4, 8:8, 9:10\}$. Simpler digits like 1 and 7 use fewer components, while visually complex digits like 3, 5, 8, and 9 use more, allowing the model to balance capacity and generalization

Both models were trained and evaluated on the same training and test datasets.

2. **Overall Accuracy:**

- Fixed-K model ($K = 10$): **95.53%**
- Variable-K model (per-class): **94.42%**

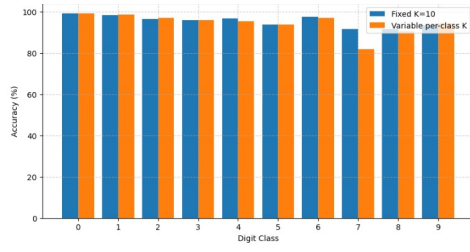
The adaptive model achieved almost the same accuracy while using approximately 40% fewer total mixture components.

3. **Per-Class Behaviour:**

- **Simple digits (0, 1):** Achieved high accuracy with only 2–3 mixtures.
- **Moderate digits (4, 6, 8):** Stable performance with 5–8 mixtures.
- **Complex digits (2, 3, 5, 9):** Required 8–10 mixtures to capture diverse handwriting variations.
- **Digit 7:** Slight drop in accuracy when limited to $K = 4$ due to occasional style variations.

4. **Insights:**

- Simpler digits with clean, single-stroke shapes can be modeled efficiently using fewer Gaussian components.
- More complex digits with multiple loops or curves benefit from a higher number of components to represent intra-class variability.
- Adaptive allocation of mixture components reduces model size and computational cost while preserving classification accuracy.



(a) Per Class Accuracy: Fixed vs Variable GMM components

Digit	Fixed-K Acc	Variable-K Acc	K used
0	99.18%	99.08%	3
1	98.33%	98.59%	2
2	96.41%	97.09%	8
3	96.04%	96.04%	10
4	96.74%	95.32%	5
5	93.83%	93.72%	9
6	97.70%	97.08%	6
7	91.54%	81.91%	4
8	91.58%	90.66%	8
9	93.56%	94.45%	10

(b) Optimal Values of K for each class

Figure 7: Comparison of per-class performance and optimal K values in GMM.

References

- [1] Lester Mackey (Instructor), “Lecture 2 — Gaussian Mixture Modelling / Expectation-Maximisation,” *STATS 306B: Methods for Applied Statistics (Unsupervised Learning)*, Stanford University, Spring 2014.
https://web.stanford.edu/~lmackey/stats306b/doc/stats306b-spring14-lecture2_scribed.pdf
- [2] Martin Haugh, “The EM Algorithm,” *IEOR E4570: Machine Learning for OR&FE*, Columbia University, Spring 2015.
https://www.columbia.edu/~mh2078/MachineLearningORFE/EM_Algorithm.pdf
- [3] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake VanderPlas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, Édouard Duchesnay, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011..
<https://jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf>